

Impact of Vector Databases on Information Retrieval for AI Agents Specialized in SDN API Documentation

Carlos Sebastian Madrigal Rodriguez

June 10, 2026

1 Introduction

As modern networks become increasingly software-defined and API-driven, automation platforms expose numerous APIs for configuration, monitoring, troubleshooting, and operational management. However, the complexity and volume of technical documentation and hardcoded API interactions make it difficult for both automation systems and network engineers to efficiently identify and utilize the appropriate APIs, limiting the effectiveness of network automation.

Retrieval-Augmented Generation (RAG) architectures provide a promising approach to this challenge by combining large language models with external knowledge sources. Through semantic retrieval, these systems can access and reason over technical documentation in real time, enabling AI agents to discover relevant APIs and support automation workflows beyond traditional keyword-based search (Lewis et al., 2020; Wu et al., 2024).

Vector databases constitute a key component of RAG architectures by enabling efficient similarity searches over embedded representations of documentation. This capability may improve the discovery of relevant technical information and facilitate contextual reasoning over large API repositories (Johnson et al., 2019).

Despite their growing adoption, the effectiveness of vector databases for technical documentation retrieval in network automation remains insufficiently understood. This research therefore investigates the use of vector databases for AI-driven retrieval of SDN API documentation and evaluates the performance of FAISS, PGVector, and Neo4j in terms of retrieval accuracy, response quality, and operational performance.

2 Materials and Methods

2.1 Hypothesis

Hypothesis 1 (Main): The incorporation of vector databases into RAG-based AI agents improves knowledge retrieval and API discovery in SDN documentation compared to traditional search methods.

Hypothesis 2: Vector database-based retrieval achieves higher retrieval accuracy than keyword-based search.

Hypothesis 3: Implementations of vector databases like FAISS, PGVector, and Neo4j exhibit statistically significant differences in retrieval accuracy, response quality, and latency.

Null Hypothesis: The choice of vector database does not significantly affect the quality of responses generated by RAG-based AI agents.

Hypothesis 1 is the base of the investigation, while Hypotheses 2 and 3 are interdependant specific sub-hypotheses that will be tested to evaluate the overall impact. It might be possible that specific algorithms may align better with a specific set of data, for which hypothesis 3 is designed to evaluate the performance of different implementations, in this case: FAISS, PGVector, and Neo4j.

2.2 Data Acquisition and Ethical Considerations

The experimental corpus was constructed from publicly available Cisco SD-WAN API DevNet documentation (Cisco Systems, Inc., 2025a) and the OpenAPI specification provided by Cisco SD-WAN vManage (Cisco Systems, Inc., 2025b).

API documentation content is processed into embeddings and stored within vector databases solely to support retrieval experiments. The original documents remain the intellectual property of their respective owners and are not redistributed as part of this research. References to all documentation sources are provided to ensure transparency and reproducibility.

2.3 Data Reconnaissance and Preprocessing

The JSON formatted API data that Cisco SD-WAN vManage OpenAPI specification (Cisco Systems, Inc., 2025b) already provides is parsed to extract relevant fields for indexing and retrieval. The main fields extracted are

summarized in Table 1.

Field	Meaning
path	API URL
method	GET/POST/PUT/DELETE
tags	Functional category
summary	Short description
description	Detailed description
operationId	Unique operation name
parameters	Input parameters
requestBody	Payload definition
responses	Output definition

Table 1: Main API data structure.

As a pre-processing task prior to vectorization, the key-value pairs are isolated and converted into natural language strings. For example, the API endpoint for retrieving device information might be transformed into a string like:

Administration - Audit Log
Endpoint: GET /auditlog/page
Operation: getStatBulkRawPropertyData
Description: Get raw property data in bulk.
Parameters:
+ query (required string)
+ scrollId (optional string)
+ count (required integer)
Returns: Audit log data.

This insures that each structured JSON entry transforms to a single string chunk capturing full context.

2.4 Data Treatment

Data will be processed through a common ingestion pipeline followed by three independent storage/retrieval backends. The ingestion pipeline will include text extraction, cleaning, and embedding generation using a consistent model (Qwen3-Embedding-4B, which is specifically designed for text embedding and ranking tasks) across all databases to ensure comparability.

Each vector database will be configured according to best practices for optimal performance and retrieval accuracy. The overall architecture of the ingestion pipeline is illustrated in Figure 1.

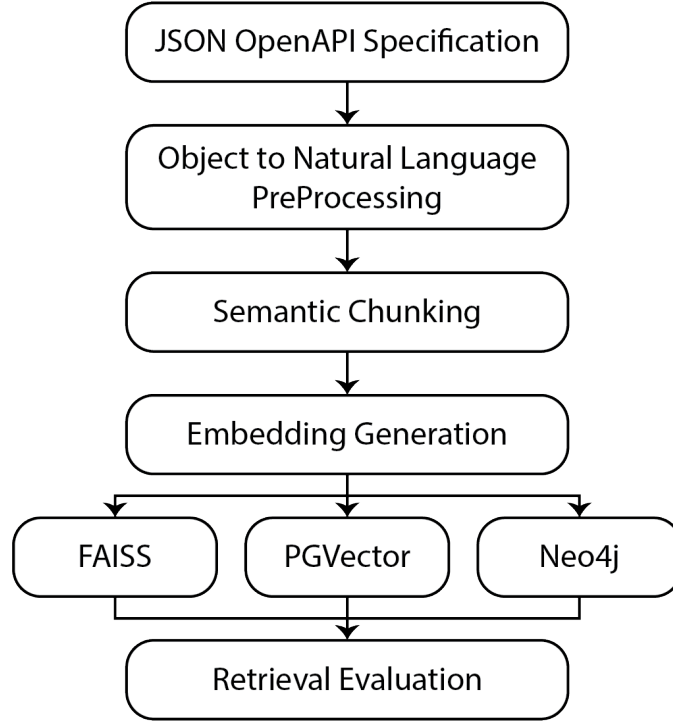


Figure 1: Ingestion pipeline for processing API data.

2.5 Evaluation Metrics

Retrieval results will be evaluated against a predefined set of queries designed to test various aspects of API discovery and information retrieval. Queries are subdivided into 3 groups and categorized based on complexity and relevance to typical network automation tasks:

- **Group 1: Basic API Discovery:** Queries that require identifying specific API endpoints or parameters.
- **Group 2: Contextual Understanding:** Queries that require understanding the context of API usage, such as dependencies or common use cases.

- **Group 3: Complex Reasoning:** Queries that require multi-step reasoning or synthesis of information across multiple documentation sections.

Each group contains 15 queries, resulting in a total of 45 queries for evaluation.

Retrieval accuracy will be evaluated on 4 different factors based on the relevance of retrieved documents to the query.

- **Precision@K:** Proportion of relevant documents in top K results.

$$\frac{\text{Number of relevant documents retrieved in top K}}{K}$$

- **Recall@K:** The proportion of relevant documents that were retrieved out of all relevant documents within the top K results:

$$\frac{\text{Number of relevant documents retrieved in top K}}{\text{Total number of relevant documents}}$$

- **MRR (Mean Reciprocal Rank):** The average of the reciprocal ranks of the first relevant document for each query:

$$\frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

where rank_i is the rank of the first relevant document for query i .

- **F1-score@K:** The harmonic mean of Precision@K and Recall@K:

$$2 \cdot \frac{\text{Precision@K} \cdot \text{Recall@K}}{\text{Precision@K} + \text{Recall@K}}$$

Response quality will be assessed through human evaluation, where the usefulness and correctness of the retrieved information are rated on a Likert scale.

Latency will be measured as the time taken from query submission to retrieval response, where automation will be used to ensure consistent timing across all tests.

References

- Cisco Systems, Inc. (2025a). *Cisco sd-wan api documentation*. <https://developer.cisco.com/docs/sdwan/overview/>
- Cisco Systems, Inc. (2025b). Cisco sd-wan vmanage openapi specification [OpenAPI specification exported from Cisco SD-WAN vManage Release 20.18.2]. Retrieved June 7, 2026, from <https://apidocs/vmanageapi.json>
- Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3), 535–547.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *arXiv preprint arXiv:2005.11401*. <https://arxiv.org/abs/2005.11401>
- Wu, S., Xiong, Y., Cui, Y., Wu, H., Chen, C., Yuan, Y., Huang, L., Liu, X., Kuo, T.-W., Guan, N., & Xue, C. J. (2024). Retrieval-augmented generation for natural language processing: A survey. *arXiv preprint arXiv:2407.13193*. <https://arxiv.org/abs/2407.13193>